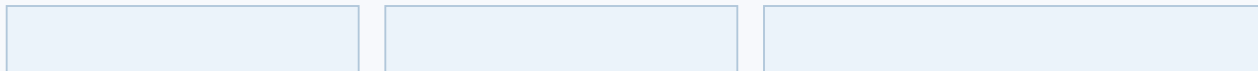


PART 5 · SOLUTION PRESENTATION

From Linux commands to resilient delivery.

A practical walkthrough of the implementation approach, architecture, evidence, decisions, and lessons.

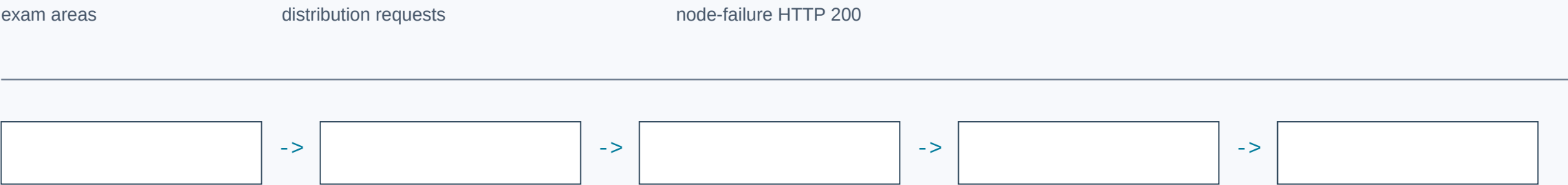
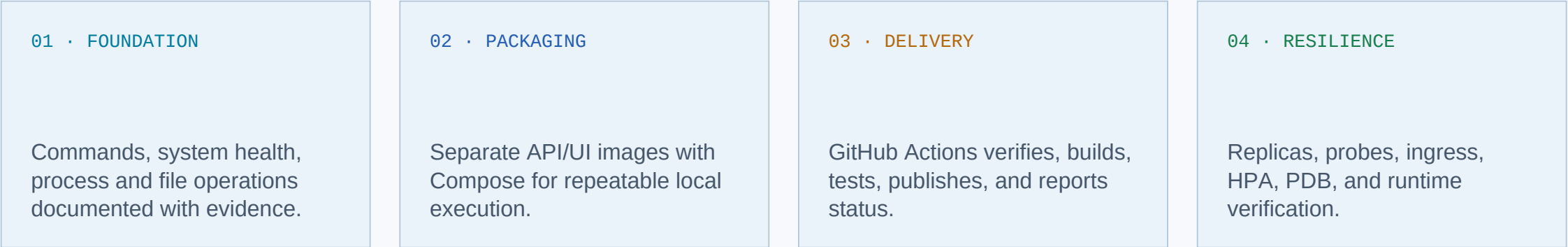


```
[ ] [ ] [ ] presentation opening
```

```
# outcome
$ git log --oneline -3
b8324a9 Kubernetes evidence
f5d58e1 Docker Hub deploy
7a51dac validation-first
build
```

EXECUTIVE SNAPSHOT

A layered solution with proof at every boundary



ARCHITECTURE

One application path, three execution environments

DOCKER-COMPOSE.YML

```
services:
  api:
    build: ./api
    depends_on: [db]
    ports: ["8000:8000"]
  ui:
    build: ./ui
    ports: ["3000:3000"]
  db:
    image: mysql:8.0

# local feedback before CI
```

LOCAL FEEDBACK LOOP

Docker Compose

API, UI, and MySQL share an isolated bridge network.

DELIVERY BOUNDARY

GitHub Actions ? Docker Hub

Tests run before publishing; SHA tags provide traceability.

RUNTIME BOUNDARY

Kubernetes / Minikube

Replicas, services, ingress, probes, HPA, and PDB.

PART 1 · LINUX FOUNDATIONS

I treated the shell as the first production interface

REPEATABLE SHELL CHECKS

```
$ bash part1-linux/system_health.sh  
CPU, memory, disk, and network  
status collected
```

```
$ bash part1-linux/log_analysis.sh  
Errors grouped by pattern
```

```
$ bash part1-linux/backup.sh  
Backup archive created safely
```

Inspect system information, packages, users, permissions, and networking.

Analyze processes and logs with standard Unix text-processing tools.

Automate health, backup, and log analysis as executable Bash scripts.

Document what each command proves.

PART 2 · DOCKER CONTAINERIZATION

The same services run locally before they ever reach CI

API/DOCKERFILE

```
FROM python:3.11-slim AS builder
RUN pip install --prefix=/install -r
requirements.txt

FROM python:3.11-slim
COPY --from=builder /install /usr/local
USER appuser
HEALTHCHECK CMD curl -f http://localhost:8000/
CMD ["uvicorn", "app.main:app"]
```

UI/DOCKERFILE

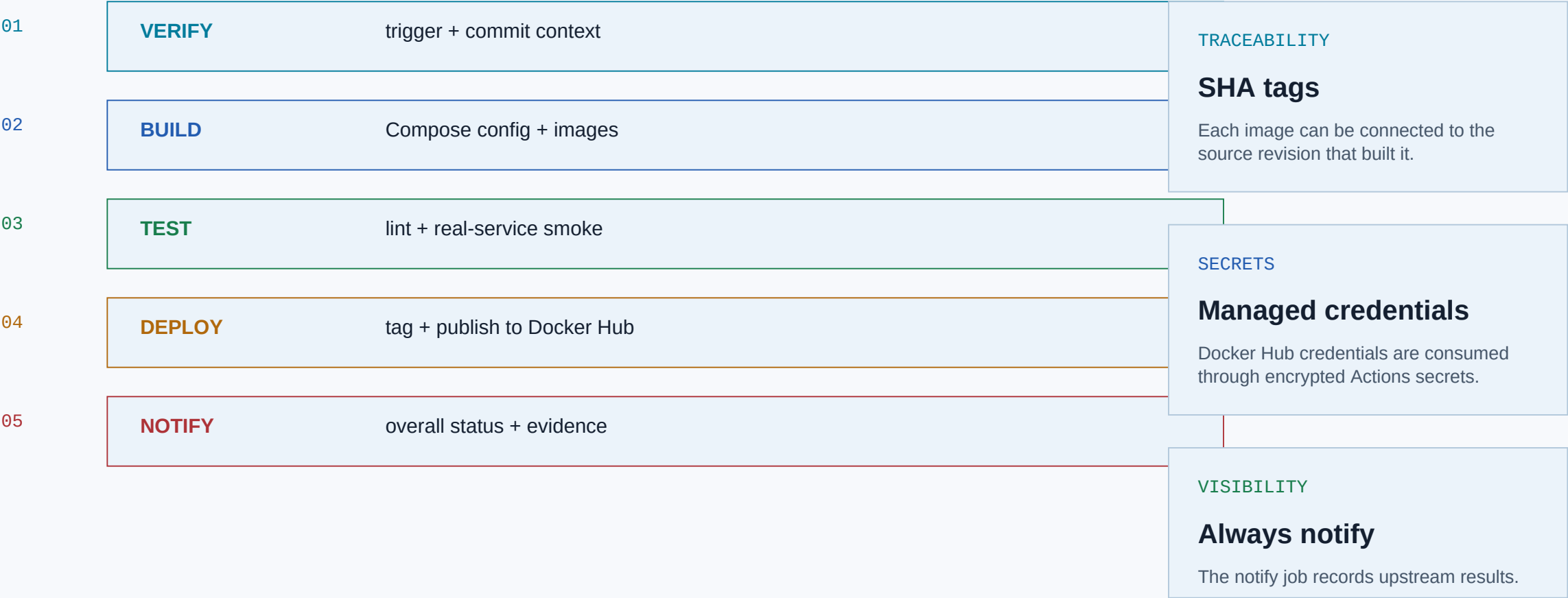
```
FROM node:20-alpine AS builder
RUN npm ci && npm run build

FROM node:20-alpine
USER nextjs
COPY --from=builder /app/.next ./next
HEALTHCHECK CMD wget -qO- http://localhost:3000/
CMD ["npm", "start"]
```

LOCAL PROOF

Compose must expose healthy API, UI, and database services before CI publishes.

PART 3 · CI/CD PIPELINE



PART 3 · EVIDENCE

The delivery path is backed by observable results

.GITHUB/WORKFLOWS/DEPLOY.YML

```
jobs:
  verify:
    steps: [checkout, compose-config]
  build:
    needs: verify
  test:
    needs: build
    # real-service smoke test
  deploy:
    needs: test
    # publish SHA + staging + latest
  notify:
    if: always()
```

CONFIRMED RESULTS

Build + test passed on GitHub-hosted runners.

Docker Hub publish completed with SHA, staging, and latest tags.

Success notification captured in the evidence trail.

PART 4 · HIGH AVAILABILITY DESIGN

REPLICATION

maxUnavailable: 0 and topology spread ask the scheduler to separate pods.

TRAFFIC

ClusterIP + Ingress provide a stable route while EndpointSlice tracks Ready backends.

FEEDBACK

Readiness gates traffic, liveness restarts, and HPA responds to load.

```
[ ] [ ] [ ] Kubernetes object model
```

DESIGN PRINCIPLE

PART 4 · EVIDENCE

The runtime checks the story the manifests promised

VERIFY-RUNTIME-EVIDENCE.SH

```
$ ./verify-runtime-evidence.sh --preflight
```

```
rollout: complete  
ready pods: 2  
expected image: confirmed  
endpoints: 2 / 2  
instance route: reachable
```

```
$ ./verify-runtime-evidence.sh  
--node-failure  
20/20 requests: HTTP 200
```

WHAT THIS PROVES

Rollout completes successfully.

At least two Ready pods with pod IPs.

All Ready pods use the expected image.

Every Ready pod IP is in the EndpointSlice.

Ingress path preserves /instance.

40 requests reached both identities within 2:1.

Captured node test: 20/20 HTTP 200.

PART 5 · THINKING AND TRADE-OFFS

The strongest decisions made failure visible

CHALLENGE ? RESOLUTION

Challenge: state from one job does not exist in the next.
Resolution: each job checks out code and declares its own setup.

CHALLENGE -> RESOLUTION

Challenge: rollout can look healthy with stale or mixed image state.
Resolution: SHA tags plus EXPECTED_API_IMAGE preflight.

CHALLENGE -> RESOLUTION

Challenge: rewrite-target turned /instance into a 404.
Resolution: remove stale annotation and verify path handling.

TRADE-OFF

Minikube is reproducible and exam-friendly, but not production control-plane durability. Next: repeat manifests in managed staging.

LIVE WALKTHROUGH RUNBOOK

A calm demo is a prepared demo

```
[ ] [ ] [ ] demo order · do not improvise
```

```
# 1. repository state
$ git status --short --branch
$ git log --oneline --decorate -3

# 2. local service contract
$ docker compose -f part2-docker/docker-compose.yml config
--quiet
? compose configuration valid

# 3. HA readiness gate
$ EXPECTED_API_IMAGE=draimon112/devops-api:staging \
  ./part4-ha/verify-runtime-evidence.sh --preflight
```

RULE 01

Show the boundary

RULE 02

Explain before output

State what the command should prove before running it.

RULE 03

Have an evidence fallback

CLOSING STATEMENT

Reliable delivery is a chain of verified boundaries

Reliable delivery is a chain of verified boundaries.

Linux makes state observable. Docker makes services portable. CI/CD makes delivery repeatable. Kubernetes makes availability testable.

WITH MORE TIME

[OK] Security gate with image scanning.

[OK] Immutable digests in deployment manifests.

[OK] A tested rollback procedure.

[OK] Production-grade observability.

Thank you · Questions welcome

SAY

Good day. I will present the solution as a delivery story: Linux and container foundations, an automated path to a registry, and a two-replica Kubernetes workload with runtime evidence.

WALK-THROUGH

WALK-THROUGH

Open the repository README and point to the four parts. Start with the outcome, not a code dump.

IF ASKED

Scope: Linux basics, Docker containerization, CI/CD, and high availability deployment.

SAY

My approach was incremental and evidence-driven. Each layer produces an artifact the next layer can consume: scripts and docs, images, a pipeline, then Kubernetes manifests and runtime verification.

WALK-THROUGH

WALK-THROUGH

```
tree -L 2  
sed -n '1,220p' README.md  
git status --short --branch
```

IF ASKED

This order reduces debugging scope: if the container is not healthy, Kubernetes is not the place to debug the application.

SAY

The key decision is separation of concerns. Compose provides local feedback, GitHub Actions provides repeatable delivery, and Kubernetes provides availability controls.

WALK-THROUGH

WALK-THROUGH

Show: `part2-docker/docker-compose.yml`, `.github/workflows/deploy.yml`, `part4-ha/k8s/`

IF ASKED

Separate services allow independent scaling, clearer health boundaries, and a smaller blast radius.

SAY

Before containers or Kubernetes, I established operational habits: inspect the system, understand processes and networking, manage files safely, and automate repeated checks.

WALK-THROUGH

WALK-THROUGH

```
bash part1-linux/system_health.sh  
bash part1-linux/backup.sh  
bash part1-linux/log_analysis.sh
```

IF ASKED

The scripts turn ad-hoc diagnosis into a repeatable baseline.

SAY

I used separate multi-stage images for the API and UI because they have different runtimes, dependencies, ports, and scaling behavior. Compose adds the MySQL dependency.

WALK-THROUGH

WALK-THROUGH

Open the two Dockerfiles and `docker-compose.yml`. Run `docker compose ... config --quiet`, then `ps`.

IF ASKED

If the local contract is unstable, the CI smoke test should fail before publishing.

SAY

The pipeline is deliberately modular: verify, build, test, deploy, and notify. Only after the test stage passes does Deploy publish Docker Hub tags.

WALK-THROUGH

WALK-THROUGH

Open `.github/workflows/deploy.yml` and point to the jobs in dependency order.

IF ASKED

Fresh runners make hidden local state visible, so each job must declare its dependencies and readiness checks.

SAY

These screenshots show successful build, test, publish, and notification proof. Optional image scanning and a live rollback exercise are follow-up work, not overclaimed as complete.

WALK-THROUGH

WALK-THROUGH

Show CI success, success email, and Docker Hub tags. Then show the current workflow file.

IF ASKED

I did not intentionally break a working pipeline just to create failure-email evidence.

SAY

Kubernetes provides the primitives the requirement needs: replicas, scheduling constraints, probes, stable Services, Ingress, autoscaling, and disruption protection.

WALK-THROUGH

WALK-THROUGH

Open `api-deployment.yaml`, `api-service.yaml`, `hpa.yaml`, `poddisruptionbudget.yaml`, and `verify-runtime-evidence.sh`.

IF ASKED

Minikube is a reproducible local exam environment; the manifests can be repeated in managed Kubernetes.

SAY

The important detail is the preflight. I do not count requests just because rollout says complete. I first verify pods, images, EndpointSlice membership, ingress behavior, and /instance.

WALK-THROUGH

WALK-THROUGH

```
EXPECTED_API_IMAGE=draiimon112/devops-api:staging ./part4-ha/verify-runtime-evidence.sh --preflight  
./part4-ha/verify-runtime-evidence.sh  
./part4-ha/verify-runtime-evidence.sh --node-failure
```

IF ASKED

A Ready pod not present in the EndpointSlice is not receiving traffic, so pod status alone can mislead.

SAY

The most useful challenges were boundary problems: fresh CI runners, mutable image tags, and ingress path behavior. I resolved them with explicit readiness, SHA traceability, and preflight checks.

WALK-THROUGH

WALK-THROUGH

Use the slide as the reflection section. Clearly separate completed work, documented strategy, and next improvements.

IF ASKED

Next improvements: image scanning, immutable deployment digests, tested rollback, and managed staging.

SAY

I will keep the demo deterministic: repository state, Compose validation, workflow shape, then Kubernetes preflight or captured evidence. I will not improvise if the cluster is unavailable.

WALK-THROUGH

WALK-THROUGH

```
git status --short --branch  
git log --oneline --decorate -3  
docker compose -f part2-docker/docker-compose.yml config --quiet
```

IF ASKED

If a command fails, stop at that boundary and explain whether it is environment readiness or application behavior.

SAY

My main takeaway is that reliability is not one tool. It is a chain of verified boundaries. I can show the repository, explain the choices, and reproduce the checks.

WALK-THROUGH

WALK-THROUGH

Finish with: I am happy to walk through any file or command in more detail.

IF ASKED

Never infer runtime correctness from one green signal; verify the boundary that carries traffic.